

MoE-LoRA for Parameter-Efficient Texture Classification

Nicholas Nevan Kurniawan
2702268441

ReynardAmadeus Joshua
2702339702

Yoel Augustan
2702215561

Abstract—Fine-tuning large-scale vision transformers requires a great amount of computation, resulting difficulties for practitioners with few resources. LoRA tackles the parameters well but makes the same changes for all inputs, which is a big problem for texture classification, as the same texture attribute might appear very differently across different datasets. Based on the context, the researcher proposes MoE-LoRA, a combination of Mixture of Experts and Low-Rank Adaptation. This approach combines experts with specialized low rank within the compute-all framework with the help of dynamic routing weights for the contribution from specialized experts for each input element. MoE-LoRA achieves Top-1 accuracy of 70.05% and Top-5 accuracy of 91.81% on the Describable Textures Dataset with only 196,287 trainable parameters (0.90% of the total). Another uniqueness of MoE-LoRA is the achievement of Top-5 accuracy of 91.81%, indicating that the ensemble method from the experts is useful for making effective predictions with the help of representation diversity. The research demonstrates that expert specialization is effective, with each expert allocating 10 to 40 percent of their time accordingly. This is why MoE-LoRA is so useful for things like interactive categorization and retrieval based on texture.

Keywords—*Mixture of Expert, Low-Rank Adaptation, MoE-LoRA, Texture Classification, Parameter-Efficient Fine-Tuning, Vision Transformers, Data-Efficient Image Transformer*

I. INTRODUCTION

Fine-tuning big models takes a lot of resources, which is a problem for researchers that has limited resources. This specific issue is important in the field of computer vision, where the most advanced Vision Transformer needs a significant amount of resources because it has a lot of parameters [1]. To address this issue, parameter-efficient fine-tuning (PEFT) techniques have emerged, including Low-Rank Adaptation (LoRA), which is highly effective as it incorporates trainable low-rank matrices into the frozen pre-trained weights, achieving performance comparable to full fine-tuning with merely 1-5% of the trainable parameters [2].

However, when LoRA is normally used, it had a set adaptation strategy that made sure all the inputs were updated. This method could not take into account how different the datasets were, which made it hard to recognize textures when the same texture properties might look very different in different visual contexts [3]. The Mixture-of-Experts (MoE) models help resolve these issues because the MoE models allow neural networks to learn task-specific representations using expert routing mechanisms [4]. However, the traditional MoE implementations suffer from parameter explosion, as each traditional expert is typically a full-rank neural network [5]. Also, optimal computational sparsity when selectively executing the experts is hard using traditional kernels and systems. This is a crucial challenge, the researchers desire dynamic, input-specific adaptation without the parameter overhead that exists with traditional expert designs.

The researchers introduce the MoE-LoRA for texture classification on the challenging DTD dataset. The researchers evaluated the strategy on the challenging Describable Textures Dataset [6], where the model had to learn to distinguish 47 texture types that were "in the wild," such as "striped" patterns on clothes, buildings, and natural surfaces, and "woven" textures on fabrics, baskets, and organic materials. The researcher selected Data-efficient Image Transformer (DeiT) [7] as our control model because the knowledge distillation training produces stronger representations than average ViT models. The researcher's hypothesis is that letting the model send different texture instances through specialized experts during training will help it tell the difference better than static LoRA adaptation. The researchers also speculate that MoE-LoRA dominates static LoRA without the overfitting risk of full fine-tuning, primarily with the stronger pre-trained representations of DeiT-Small.

This work meets real-world needs in material science and quality control, where automated texture recognition can be used to perform surface inspections, classify materials, and detect defects. Computer vision systems are also applied to interior design, fashion recommendations, and image retrieval, all applications where robust texture understanding is important. Other applications involve consumer-facing scenarios, such as material identification or other DIY projects for educational apps.

II. RELATED WORK

Vaswani et al. [8] proposed the Transformer model that transformed the sequence modeling by using self-attention exclusively rather than using recurrence or convolution. Its key invention, multi-head self-attention, projects the input into query (Q), key (K), and value (V) matrices to calculate the contextual relationships for all positions. Its architecture has state-of-the-art performance for the machine translation while it is very parallelizable that sets the stage for the current transformer models for NLP and computer vision.

Dosovitskiy et al. [1] extended transformations to vision work by splitting images into fixed-length patches and tokenizing them for sequence processing. Their Vision Transformer (ViT) showed that end-to-end transformer models alone can match or beat CNNs when trained from enormous datasets. But because pre-training the ViT required large-scale data (e.g., JFT-300M), its accessibility to smaller research groups was hindered. Touvron et al. [7] closed this data ineffectiveness with DeiT, a data-efficient variant of the ViT that is trained end-to-end on the mere ImageNet-1k without additional data using teacher-student distillation by a designated distillation token. The model reaches 85.2% ImageNet top-1 accuracy using no additional data, rendering it a popular choice for parameter-efficient fine-tuning.

Hu et al. [2] introduced LoRA to save fine-tuning expenses by freezing pre-trained weights and adding trainable low-rank upgrade matrices. LoRA attains performance equivalent to

complete fine-tuning using just 1 until 5% trainable parameters and lower GPU memory utilization by a factor of three. It, however, uses homogeneous transformations to all instances without much flexibility to model input-specialized variations, a crucial limitation for tasks with high intra-class variation. Shazeer et al. [4] came up with sparsely gated MoE doors where a trainable gating network wakes up a small subset of experts per input. MoE has the potential to have humongous model capacity with negligible computational overhead but the expense is highly in terms of parameters because each expert is a full-rank network.

Wu et al. [9] introduced MoLE for multi-task learning by routing between a number of pre-trained LoRA adapters via a gating network. This allows a model to process varied tasks in an efficient manner but is not trained for sample-level adaptation across a single dataset. Dou et al. [10] proposed LoRAMoE to avoid catastrophic forgetting for large language models, by using a number of LoRA experts to maintain various aspects of world knowledge through world dynamics routing across tasks. Luo et al. [11] further extended this work by steering LoRA expert specialization using contrastive learning to promote unique, input-dependent feature adaptation for NLP. Gao et al. [12] examined expert placement and discovered mid-to-upper transformer levels are most improved by LoRA experts, implying that focused adaptation is more efficient.

Cimpoi et al. [6] created the Describable Textures Dataset, which has 5,640 photos sorted into 47 human-centric texture characteristic categories (including "striped," "dotted," "woven," and "braided") with 120 images per class. The photographs were taken "in the wild" from Google and Flickr. The dataset presents significant challenges due to extreme intra-class variation, as the same texture attribute can appear across vastly different materials, scales, orientations, and lighting conditions, for instance, "striped" patterns manifesting on clothing, architecture, animals, and natural surfaces with drastically different visual characteristics.

Cimpoi et al. [13] proposed FV-CNN, Fisher Vector pooled CNN features, a combination of Fisher Vector encoding and deep convolutional features from VGG networks to build powerful texture representations. Their approach reached state-of-the-art performance on DTD and multiple classical benchmarks of texture recognition by proving that deep features pooled with classic representations like Fisher Vectors outperform by a big margin traditional handcrafted texture descriptors. Simon et al. [14] proposed DeepLumina, which enhances the texture classification task by combining deep features from CNNs and luminance information from the YIQ color model. It extracts features from pre-trained ResNet101 and incorporates luminance channel (Y) information along with RGB color space, performing a classification through Support Vector Machines. DeepLumina achieves 73.63% on DTD using its approach.

III. METHODOLOGY

A. Data Preprocessing and Augmentation

The training data pipeline performs several augmentation techniques to further improve robustness and generalization capabilities. First, a random resize of the image with the value between 0.5 and 1.0 and random cropping is applied, allowing the model to see textures at various scales and positions, simulating different viewing distance and allows the model to learn the features. Random horizontal flips are performed to

further augment the number of photos for training and let the model learn that a texture does not change if it was mirrored, this was helpful for nondirectional patterns, such as dotted or meshed textures. Then the TrivialAugmentWide augmentation is also used to perform changes such as color jittering, contrast adjustment, brightness modification, and sharpness changes [15]. This helps the model learn on different lighting and camera settings. Finally, all the augmented photos are resized to 224×224 pixels to follow Vision Transformers' criteria for input.

All images were then transformed into tensors and normalized using ImageNet statistics, bringing data closer to zero and equalizes variances to one. This approach also provides more stable training and speeds convergence up. The validation and test data also went through deterministic preprocessing so the evaluation is identical. The images were scaled to 256×256 pixels and center-crops to 224×224 pixels. It is also important that there were no random augmentations during the evaluation, to ensure trustworthy and comparable performance measures among models. Mixup and CutMix are also employed during training to help generalize. Mixup [16] generates synthetic samples by linearly mixing pair-wise pictures with their labels and promoting more fluid decision boundaries. CutMix [17], on the other hand, cuts and pastes rectangular patches between photos and mixes the labels based on patch regions. An important aspect that needs to be noted is that even though both Mixup and CutMix are used uniformly during the process of fine-tuning, the pre-trained models of the backbones have different original ImageNet pre-training augmentation recipes, which is necessary for the performance during the downstream task. DeiT-Small requires an aggressive augmentation technique during pre-training, while ViT-Small requires a lighter augmentation technique.

B. Based Model Architecture

The key models the researchers mainly utilize are Vision Transformer [18] and Data-efficient Image Transformer [7]. These transformer-based models break the input images into patches, linearly embed them, then input them into numerous levels of multihead self-attention. This structure allows the model to discover long-range dependencies while showing complex spatial correlations between texture patterns.

In particular, the ViT-Small backbone employed in the current study is pre-trained using ImageNet-1k with the AugReg "medium2" recipe. While moderately regularizing and augmenting the training relative to vanilla ViT training, this recipe does not contain aggressive example-level augmentations like CutMix and repeated augmentation. In contrast, DeiT-Small employed in the current study does not make use of a distillation token when fine-tuning. In fact, it strictly follows the original DeiT training recipe, where the primary focus is put on performing data-efficient learning through the use of strong augmentations such as Mixup, CutMix, stochastic depth, and repeated augmentation while performing ImageNet pre-training.

The researchers then look at and compare four different fine-tuning methods, each of which has its own pros and cons when it comes to how well it works and how well it uses the parameters. These methods include changing all the parameters and only training the parts that are specific to the task, which gives a complete picture of how to change texture classification in the best way.

1) Full Fine-Tuning

In this baseline method, all of the pre-trained model's parameters are changed and unfrozen during training. The classification head is replaced with a new linear layer matching the 47 texture classes and both the backbone and head are optimized together. This method sets the highest level of performance for the model, but it needs to update millions of parameters and a lot of computing power.

2) Linear Probe

This procedure only trains the classification head and all the backbone parameters stay the same. This is the most efficient way to use parameters. It tests how well pre-trained ImageNet features work for texture classification and sets the lowest limit for comparison. Linear probing is simple, yet it can work well when the pre-trained features match the target job well.

3) Low-Rank Adaptation (LoRA)

LoRA [2] provides a parameter-efficient method by incorporating small trainable adapters into static pre-trained weights. LoRA doesn't change the original weight matrix W directly. Instead, it learns a different low-rank update that is added to the calculation. The researchers split this update into two smaller matrices, A and B .

LoRA adds to the weight matrix W with dimensions $d \times k$:

$$\Delta W = BA$$

Where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

The forward pass calculates:

$$h = Wx + (\alpha/r) \cdot BAx$$

Matrix A is initialized with Kaiming uniform and B starts at zero, making sure that there is no initial adaptation. The researchers use LoRA on query-key-value (QKV) projection layers in some attention blocks. This helps focus on specific textures while keeping general features.

4) Mixture of Experts with LoRA (MoE-LoRA)

The researchers suggest MoE-LoRA, a new architecture which combines the parameter efficiency of Low-Rank Adaptation with the representational diversity of Mixture of Experts. Contrary to the conventional sparse MoE models which focus on the computation efficiency using sparse execution, MoE-LoRA focuses on learning distinctive texture representations via a vectorized compute-all-route-k strategy. There are three main parts to the architecture: Router Network, Batched of LoRA experts, and auxiliary load-balancing loss.

a) Router Network

For the input token $x \in \mathbb{R}^d$, the router computes:

$$G(x) = \text{softmax}(Wx + b)$$

where $W \in \mathbb{R}^{N \times d}$ is the learned routing matrix. The router then selects the top-k experts with the highest routing probabilities, as opposed to carrying out only the selected experts, the router computes weights that mix contributions from all experts, where the weights for unselected experts are close to zero.

b) Batched LoRA Experts

To maximize parallelization efficiency when using modern accelerators, all N experts work simultaneously on a dense vectorized processing of their tokens. Each expert i has

two low-rank matrices (A_i , B_i) and the forward pass computes:

$$\Delta h_i = B_i A_i x \cdot (\alpha/r)$$

for all of the experts at the same time, using efficient tensor operations. Then, the routing weights are used to combine the outputs from all the experts:

$$\Delta h = \sum g_i \cdot \Delta h_i$$

where g_i is the routing weight for expert i . Although g_i is close to zero for non-selected experts (effectively excluding their contribution), the compute-all approach computes all experts on the forward pass. The compute-all-route-k method is focused on simplified implementation and dense computations on the GPU rather than computational sparsity and still provides training stability with gradients for the experts. This increases the computations very slightly due to the low rank structure that holds the total LoRA computations minimal in comparison with the self-attention of the backbone.

c) Auxiliary Load Balancing Loss

To prevent expert collapse, where the router learns to always favor a small group of experts and ignore the others, The researcher use auxiliary load-balancing loss,

$$L_{aux} = N \cdot \sum (f_i \cdot P_i)$$

where N is the number of experts, f_i is the percentage of tokens given to expert i (based on top-k selection), and P_i is the average routing probability given to expert i across all tokens. When the routing probabilities and the actual token assignments are evenly spread out across experts, the loss is at its lowest. The auxiliary loss is added to the primary classification loss with a weight coefficient that may be changed. This makes a multi-objective optimization that finds the right balance between task performance and expert use.

C. Architecture Overview

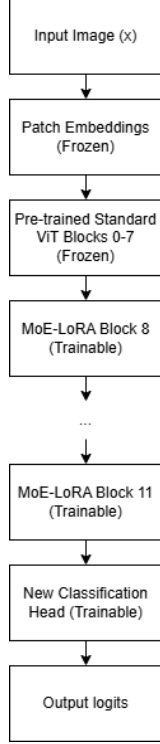


Fig. 1. Architecture Macro View

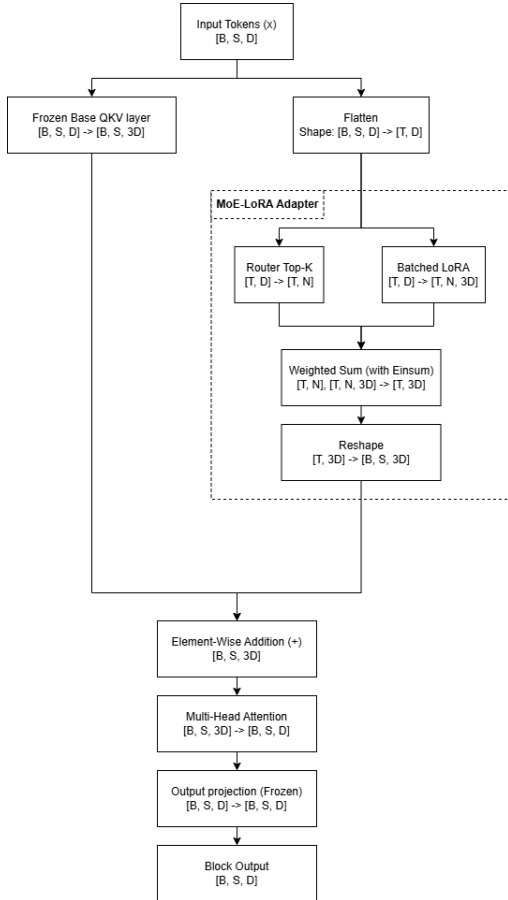


Fig. 2. Architecture Micro view (per-modified block)

In figure 1, the macro architecture of the MoE-LoRA model is shown. In this, the input image is split into patches and then embedded using frozen patch embeddings. The embeddings are then passed through pre-trained standard ViT blocks from 0 to 7, and these blocks are frozen during the fine-tuning process. The MoE-LoRA blocks from 8 to 11 are then applied, and dynamic expert routing is carried out. In the final step, a new classification head generates the output logits for the 47 classes of texture.

Figure 2 depicts the micro-architecture of MoE-LoRA blocks. The input tokens are processed via two channels: the frozen base QKV layer, which is routed, and a flattened representation of routing. The MoE-LoRA Adapter module includes a Router, which implements Top-K routing, as well as a Batched LoRA module, which processes tokens from all experts at once. The weighted sum of experts is computed based on routing, and it is summed element-wise with the output of the frozen base layer after applying multi-head attention.

D. Training Configuration

The AdamW optimizer with decoupled weight decay regularization is used to optimize all models. This has been demonstrated to help transformer systems generalize better. The learning rate is adjusted for each training method and set up in two steps. Firstly, an optional warmup period is used to increase the learning rate linearly from almost zero to the target value. This is done to keep things from getting unstable at the beginning of training. Next is cosine annealing, which slowly lowers the learning rate so that the model can be fine-tuned and reach sharper minima. To keep the gradient from exploding, the researchers use gradient clipping with a maximum norm cutoff. This is especially important for MoE structures that have multiple computational path. Finally, the training is monitored with early stopping and patience, training stops if the validation loss doesn't stop improving for a set number of consecutive epochs and the best checkpoint is then kept for testing.

Afterwards, a few regularization techniques are used to make generalization better. Label smoothing changes hard one-hot labels into soft distributions. This helps stopping the model from becoming overconfident. Then Mixup and CutMix add fake training samples by swapping out patches and filling in the spaces between them. This is especially useful for classifying textures because it helps the model find them even when they are mixed together or only partially visible. The strength of the interpolation is controlled by the Mixup coefficient. In addition, dropout in the transformer's attention and feedforward layers adds a bit of stochastic noise to the model, stopping features from becoming too dependent on each other and encouraging the model to produce more robust representations.

E. Evaluation Metrics

The researcher use Top1 and Top5 accuracy to determine how well the model perform and use this metrics on the training, validation and test sets to see if the model is overfitting and to see how well it generalize. The validation loss, its then use to pick the best model and also for early stopping. Parameter efficiency is measured as the number of trainable parameters out of the total number of parameters, and MoE-LoRA gives a breakdown of trainable parameters in the experts and routing networks. Then a assessment script

makes five predictions for each test sample and can also store the expert diagnostics. Expert indices and routing weights, for MoE-LoRA models, lets the researcher take a deeper look at how the experts specialise.

IV. RESULT & IMPLEMENTATION

All experiments are done with PyTorch and trained with GPU. The *timm* library has basic implementations of Vision Transformer and DeiT with ImageNet weights that have already been trained. The *peft* library from Hugging Face is used for conventional LoRA implementation. The researchers constructed their own MoE-LoRA from the ground up using PyTorch so they could have fine-grained control over routing and expert computation. To make sure that the results can be repeated, random seeds are set in all libraries, and deterministic algorithms are enable while the cuDNN benchmark is disable. All experimental artifacts, including training logs, checkpoints, configurations, and training histories, are retained in a systematic way. The best checkpoint with its validation loss is kept together with other metadata that goes with it.

TABLE I
RESULTS FOR DEiT-SMALL BACKBONE

Method	Trainable Params	Trainable Pct (%)	Top-1 Acc (%)	Top-5 Acc (%)
Linear Probe	18,095	0.08	68.19	89.79
Static Lora	214,703	0.98	70.96	91.44
Moe-Lora	196,287	0.90	70.05	91.81
Full Fine- tune	2,1683,759	100.0	72.71	91.70

TABLE II
RESULTS FOR ViT-SMALL BACKBONE

Method	Trainable Params	Trainable Pct (%)	Top-1 Acc (%)	Top-5 Acc (%)
Linear Probe	18,095	0.08	65.80	88.72
Static Lora	214,703	0.98	69.63	92.07
Moe-Lora	196,287	0.90	69.68	91.44
Full Fine- tune	2,1683,759	100.0	72.87	91.33

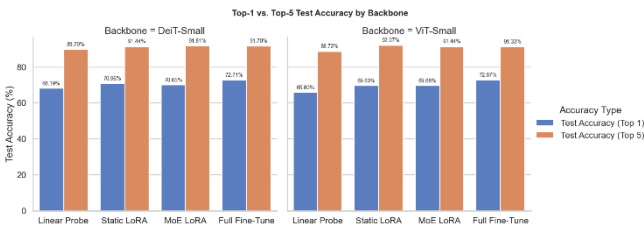


Fig. 3. Top-1 Test Accuracy

Tables I and II present the result for DeiT-Small and ViT-Small backbones respectively. Figure 3 visualize the accuracy between the four procedures. Based on the tables and graph, Full fine-tuning achieves the highest Top-1 accuracy, with 72.71% and 72.87% for DeiT-Small and ViT-Small respectively. Linear Probe achieves 68.19% on DeiT and 65.80% on ViT. Static LoRA reaches 70.96% on DeiT and 69.63% on ViT. Finally, MoE-LoRA achieves 70.05% on DeiT and 69.68% on ViT. For Top-5 Accuracy, Static LoRA and MoE-LoRA achieve the highest accuracy with 92.07% on ViT and 91.81% on DeiT respectively. Linear Probe still remains the lowest accuracy with 89.79% on DeiT and 88.72% on ViT. In contrast, Full Fine-Tune achieve 91.70% on DeiT and 91.33% on ViT, demonstrating a competitive result but require more trainable parameters.

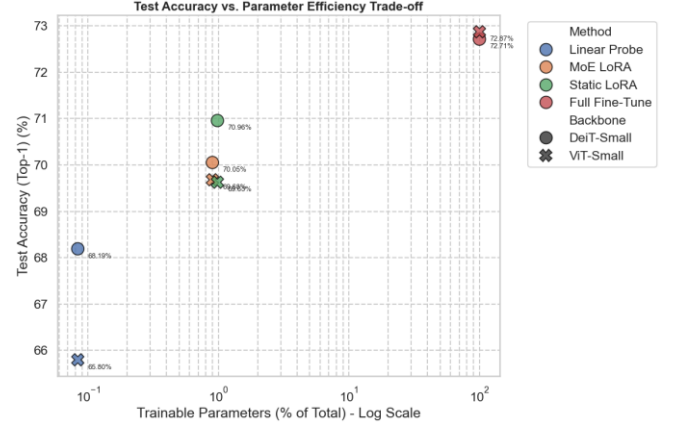


Fig. 4. Test Accuracy and Performance Trade Off

Figure 4 shows the performance-efficiency trade-off between the procedures. Linear Probe achieves the lowest accuracy with the highest efficiency. On contrast, Full Fine-Tune achieves the highest accuracy but has the worst efficiency. MoR-LoRA and Static LoRA shows a strong balance between performance and efficiency, achieving accuracies close to the Full Fine-Tune approach. Static LoRA perform slightly better, achieving 70.96%, while MoE-LoRA still demonstrate competitive result at 70.05%.

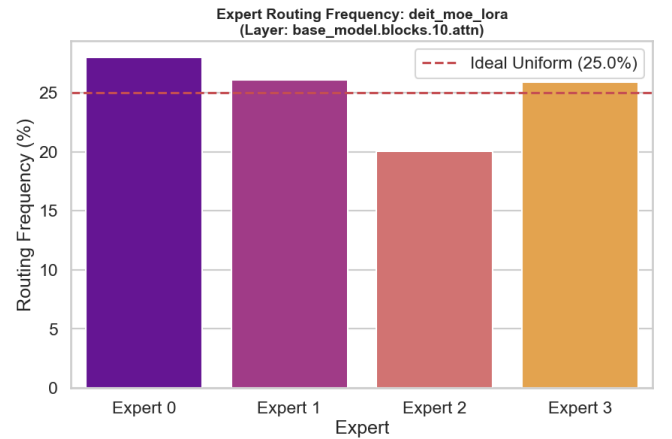


Fig. 5. Expert Routing Frequency on DeiT-Small Block 10

Based on Figure 5, the model shows balanced expert utilization in Block 10, with Expert 0 processing approximately 27% of tokens and Experts 1,2, and 3 each handling 20 to 25%. However, not all layers have the same distributions. Earlier layers, Block 8 and 9, and the deepest

MoE layer, Block 11, shows a bigger imbalance. For instance, in Block 8, Expert 0 and Expert 3 are used approximately 37% while Expert 2 handles only 10%. In Block 11, Expert 1 is utilized the most at approximately 37% while Expert 0 is utilized the least at 14%.

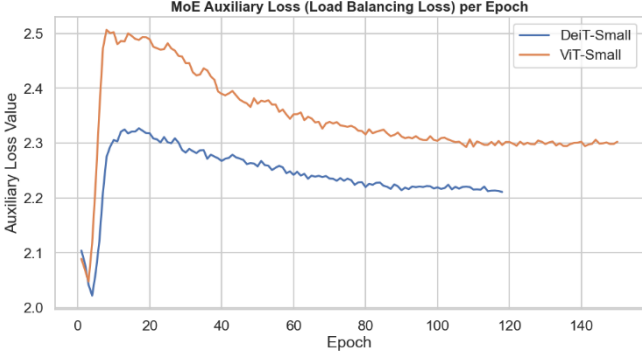


Fig. 6. Auxiliary Loss

Figure 6 tracks the auxiliary load-balancing loss for MoE-LoRA. Throughout the training ViT shows higher auxiliary loss than DeiT. Both backbones, initially increase from 2.1 to 2.5 for ViT and 2.1 to 2.3 for DeiT. Then both backbones decrease gradually and stabilize around 2.3 for ViT and 2.2 for DeiT.

V. DISCUSSION & LIMITATION

Full Fine-tuning gets the best Top-1 accuracy and competitive result for Top-5 accuracy. However this method requires to update all 21.68 million trainable parameters, which need high computational cost and memory. This approach also need to store and update the whole model, resulting in longer training, backpropagation uses more memory, and keeping several task-specific models takes up more space. This approach also make it impossible in to deploy on limited resources and edge devices, or situations where dozens of specialized models are needed at the same time.

Linear Probe, on the other hand, only needs 18,095 trainable parameters, which makes it very light and quick to train. Linear Probe uses the transferable features that were learned during ImageNet pre-training by freezing the whole pre-trained backbone and only training the last classification head. However, this method has the lowest accuracy of all the methods. The frozen backbone can't change its internal representations to fit the texture-specific patterns, so it has to rely on the linear classifier to understand fixed features that were optimized for general object recognition instead of texture-specific discrimination.

LoRA-based methods find the best balance by getting good performance without updating all of the parameters. Both Static LoRA and MoE-LoRA used about 1% trainable parameters, Static LoRA gets 70.96% Top-1 on DeiT (just 1.75% below fine-tuning), and MoE-LoRA gets 70.05% (2.66 % below fine-tuning). This LoRA approaches do surprisingly well, with the highest Top-5 accuracy on both backbones. Static LoRA achieve the highest accuracy with 92.07% on ViT and MoE-LoRA achieve 91.81% on DeiT, both of which better than full fine-tuning's Top-5 performance. This higher Top-5 accuracy and only about 2% Top-1 difference shows that low-rank adaptation works. LoRA methods use low-rank decomposition to selectively update attention weight matrices, which focuses adaptation capacity on the most important parts

for texture recognition and while leaving less important parameters the same. This targeted technique reduces the number of parameters by 100 times with very little loss of accuracy. This makes LoRA-based methods very useful for real-world use, where model efficiency is very important.

The balanced distribution in Block 10 (Expert 0 with 27% and others with 20–25%) shows that the auxiliary load-balancing loss works to stop expert collapse. The more obvious imbalances in Blocks 8, 9, and 11 show that it learned specialization rather than expert collapse, showing that the routing system lets the expert to specialise on different types of features. Importantly, all experts stay active with utilization between 10–40% even if the layers are imbalanced.

The first increase in the auxiliary loss is while the router learns to make correct decisions within the first few training sessions, then it goes down becoming more stable and proves the experts receive balanced utilization. The bigger auxiliary loss of ViT shows that this model builds stronger routing preferences compared to DeiT. This indicates that it is more difficult for this model and necessitates more stabilization on the use of experts for the model to use them equally. While ViT was performed only using supervised methods. Therefore, its internal representations became more specialized and naturally favoured some paths of experts.

It is important to mention here that the results achieved are dependent not only on the fine-tuning approach, but also on the quality of the pre-trained backbone. In this work, DeiT-Small and ViT-Small have different pre-training recipes on ImageNet, with only ViT-Small making use of the AugReg medium2 recipe, with lighter augmentation. Hence, DeiT-Small is more robust to the intra-class variation observed in texture-specific datasets. Thus, LoRA's good performance on DeiT-Small may be attributed, in part, to its robust pre-training augmentation and regularizations such as Mixup, CutMix, stochastic depth, and Repeated Augmentation. Notably, even though the DeiT series of models is commonly associated with the use of knowledge distillation, the DeiT model used in this study does not use the distillation token during the fine-tuning process, which clearly indicates that the benefits reaped differ due to the variation in the strength of augmentations alone and not due to any differences in the distillation architecture itself.

Although MoE-LoRA demonstrates a high level of parameter reduction, the computational cost of inference is marginally higher than that of Static LoRA since all LoRA experts are evaluated for a particular input token. In the researchers setting, where $r = 8$ and $d = 384$ for DeiT-Small, it adds a negligible 2.6% overhead in FLOPs along the LoRA transfer learning phase, although it is negligible when compared to a frozen transformer encoder because LoRA is a supplementary transformer encoder and consumes a negligible portion of computation overhead. Contrary to a typical sparse MoE approach where computationalateness is emphasized by using intricate routing mechanisms and specialized kernels, MoE-LoRA takes a compute-all, route-k approach, promoting simplicity and stability during training by using dense GPU-oriented computations. Nonetheless, while this is a very small overhead, this specialization is beneficial in terms of boosting representation diversity, as evident from MoE-LoRA's improved Top-5 accuracy scores compared to Static LoRA.

However, there are some limitations with the researcher's study. To start, the researchers only used one dataset and small vision transformer to evaluate MoE-LoRA on texture classification. The researchers don't know if MoE-LoRA performs well for other tasks, such as scene understanding or multi-label recognition. Texture classification involves relatively homogeneous visual analysis because all the images need to be looked at in the same way. This means that having many specialists may not display all of the benefits. The researcher also only test on small models, so there is a gap on how MoE-LoRA works with bigger or smaller models, or if the ideal settings change when the model size varies.

Furthermore, the researchers kept the same settings for all of the tests, there were always four experts and each token went to the top two experts. The researcher didn't test varying quantities of experts, such as 2, 8, or 16, or alternative routing strategies. The researcher also only contributed experts to the attention layers and not to any other portions of the model. Although our findings indicate that various experts manage differing volumes of data, it still remain uncertain about the specific areas of expertise of each expert, whether they concentrate on certain texture categories or more abstract concepts. Lastly, the researcher checked how fast the model runs during testing, but we didn't check how long it takes to train or how much memory it uses, which could be more than Static LoRA because of the routing technique.

VI. CONCLUSION & FUTURE WORK

This paper introduces MoE-LoRA, a parameter-efficient method that merges Mixture of Experts with Low-Rank Adaptation for texture classification. This technique was work on the Describable Textures Dataset (DTD) and achieves 70.05% Top-1 accuracy and 91.81% Top-5 accuracy on DeiT-Small with just 196,287 parameters (0.90% of total), showing how dynamic routing in experts allows for representation specialization with minimal parameter overhead as in standard multi-head models. MoE-LoRA's better Top-5 accuracy at 91.81% vs. 91.44% means that different experts can see different patterns in the texture. It will be very useful for systems such as texture-based image retrieval, materials identification, and interactive classification tools, which require a variety of textures to be shown. Based on the experts route traffic, the load balancing works and prevent expert collapse happens, all experts keep a utilization rate of 10–40% across transformer layers, which makes adaptive specialization easier. Deeper layers show clearer expert preferences, showing that the routing mechanism adapts to the feature that are unique to each layer.

Future work should be dedicated to several critical paths in order to enhance the overall applicability and practical utility of the model. Such as, testing MoE-LoRA on various visual recognition tasks, such as scene understanding, object detection, and multi-label recognition, along with different model sizes. Also, the researcher needs to test different hyperparameters-like changing the number of experts (2, 4, 8, or 16), trying different routing algorithms (soft routing or threshold-based selection), and finding the best way to assign experts to each layer. The researchers could also consider adding MoE to feedforward networks by making hybrid attention-FFN systems or adaptive routing systems to see if they make the model work better and give it more power.

Activation visualization and systematic probing could also be looked at, to show what features does each expert captures and if their areas of expertise follow semantic patterns. Finally, a full analysis of the training overhead including throughput, memory usage, and total training time compared to Static LoRA would give practitioners the necessary information to make decisions about which parameter-efficient fine-tuning methods to use.

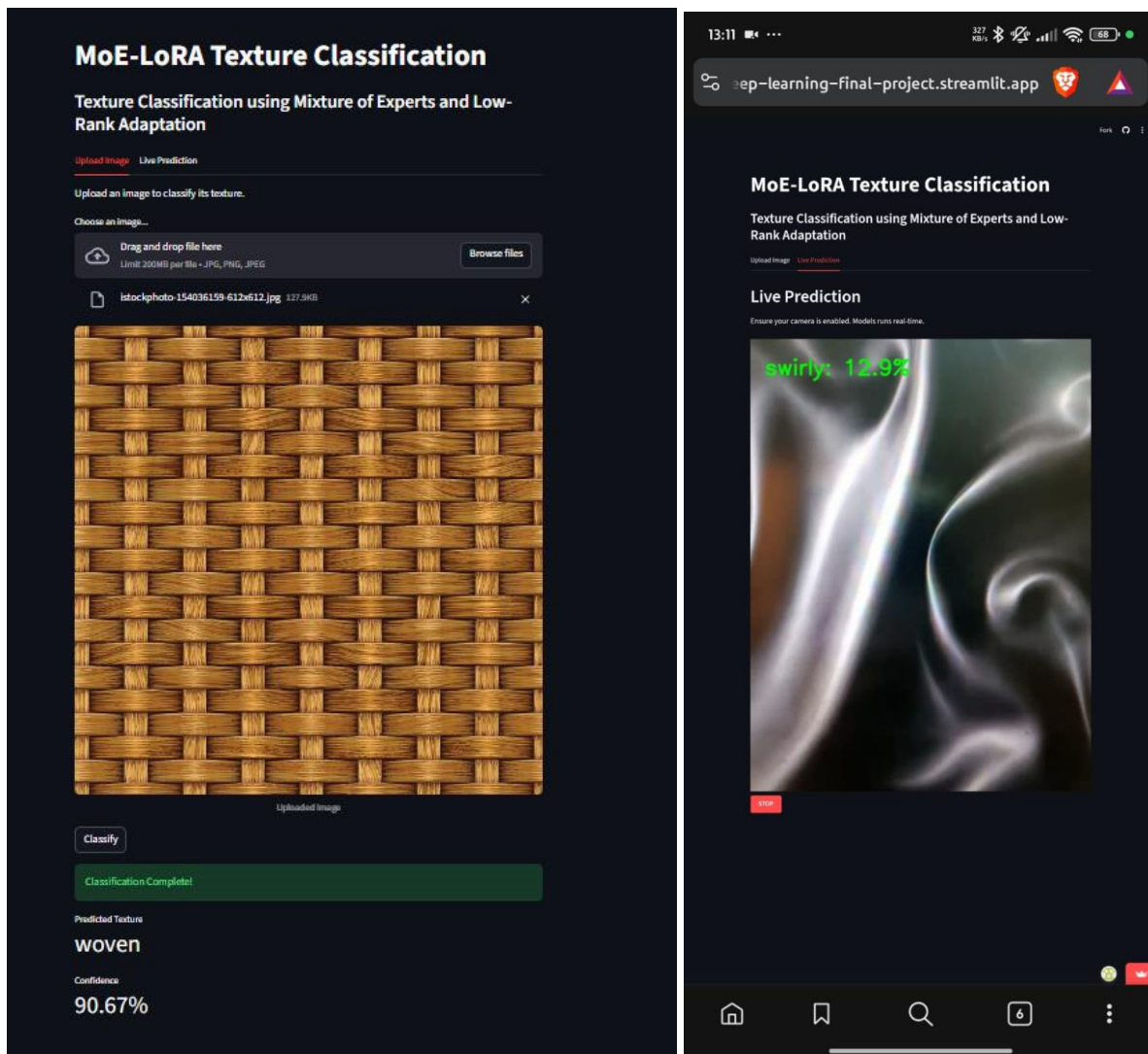
REFERENCES

- [1] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," Jun. 2021, [Online]. Available: <http://arxiv.org/abs/2010.11929>
- [2] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," Oct. 2021, [Online]. Available: <http://arxiv.org/abs/2106.09685>
- [3] X.-S. Wei *et al.*, "Fine-Grained Image Analysis with Deep Learning: A Survey," Nov. 2021, [Online]. Available: <http://arxiv.org/abs/2111.06119>
- [4] N. Shazeer *et al.*, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," Jan. 2017, [Online]. Available: <http://arxiv.org/abs/1701.06538>
- [5] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," 2022, [Online]. Available: <http://jmlr.org/papers/v23/21-0998.html>
- [6] M. Cimpoi, S. Maji, I. Kokkinosécole, S. Mohamed, and A. Vedaldi, "Describing Textures in the Wild," [Online]. Available: <http://www.robots.ox.ac.uk/>
- [7] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," Jan. 2021, [Online]. Available: <http://arxiv.org/abs/2012.12877>
- [8] A. Vaswani *et al.*, "Attention Is All You Need," Aug. 2023, [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [9] X. Wu, S. Huang, and F. Wei, "Mixture of LoRA Experts," Apr. 2024, [Online]. Available: <http://arxiv.org/abs/2404.13628>
- [10] S. Dou *et al.*, "LoRAMoE: Alleviate World Knowledge Forgetting in Large Language Models via MoE-Style Plugin," Mar. 2024, [Online]. Available: <http://arxiv.org/abs/2312.09979>
- [11] T. Luo *et al.*, "MoELoRA: Contrastive Learning Guided Mixture of Experts on Parameter-Efficient Fine-Tuning for Large Language Models," Feb. 2024, [Online]. Available: <http://arxiv.org/abs/2402.12851>
- [12] C. Gao *et al.*, "Findings of the Association for Computational MoLA: MoE LoRA with Layer-wise Expert Allocation," [Online]. Available: <https://github.com/GCYZSL/MoLA>
- [13] M. Cimpoi, S. Maji, and A. Vedaldi, "Deep Filter Banks for Texture Recognition and Segmentation," [Online]. Available: <http://www.robots.ox.ac>
- [14] A. P. Simon and B. V. Uma, "DeepLumina: A Method Based on Deep Features and Luminance Information for Color Texture Classification," *Comput Intell Neurosci*, vol. 2022, 2022, doi: 10.1155/2022/9510987.
- [15] S. G. Müller and F. Hutter, "TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation," [Online]. Available: <https://github.com/autotml/trivialaugment>
- [16] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," Apr. 2018, [Online]. Available: <http://arxiv.org/abs/1710.09412>
- [17] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features," [Online]. Available: <https://github.com/clovaai/CutMix-PyTorch>
- [18] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer, "How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers," Jun. 2022, [Online]. Available: <http://arxiv.org/abs/2106.10270>

- Team Contribution Statement

Team Contribution Statement	
Name	Job Desk
Nicholas Nevan Kurniawan - 2702268441	Create the Model
ReynardAmadeus Joshua - 2702339702	Create the Web Application
Yoel Augustan - 2702215561	Create the Paper

- App Screenshot



- Code Snippet

<https://github.com/nevan-kurniawan/deep-learning-final-project>


```

"""
This module defines the ModifiedModel class.

The module defines the ModifiedModel class as the primary orchestrator, and
the MoE_LoRA class is the implementation for the MoE-LoRA architecture.
The MoE_LoRA class uses the Router and BatchedLoRA
module as part of its logic.
"""

import torch
from torch import nn
from torch.nn import functional as F
from peft import LoraConfig, get_peft_model
import math

class Router(nn.Module):
    """
    Implements a top-k gating mechanism (router) for a Mixture-of-Experts layer.

    This module takes token embeddings as input and outputs the indices and weights
    for the top-k experts, along with an auxiliary loss for load balancing.
    """
    def __init__(self, input_dim: int, num_experts: int, k: int = 2):
        """
        Initializes the Router module.
        Args:
            input_dim: The embedding dimension of the input tokens.
            num_experts: Total number of experts (N).
            k: Total number of experts to route tokens to (default is 2).
        """
        super().__init__()
        self.k = k
        self.num_experts = num_experts
        self.gating_network = nn.Linear(input_dim, self.num_experts)

    def compute_auxiliary_loss(self, router_probs: torch.Tensor, expert_mask: torch.Tensor) → torch.Tensor:
        """
        Computes the auxiliary load-balancing loss.

        The auxiliary load-balancing loss is used to prevent expert collapse, where the model favors a few experts,
        and learns to route only to those experts, leaving the rest under-trained. This loss ensures the tokens are routed
        relatively evenly to all experts.
        Args:
            router_probs: The softmax probabilities output by the gating network. Shape: [T, N] (num_tokens, num_experts).
            expert_mask: A boolean mask indicating which tokens were dispatched to which expert. Shape: [T, N] (num_tokens, num_experts).

        Returns:
            auxiliary_loss: Scalar auxiliary loss.
        """
        if router_probs is None or expert_mask is None:
            return torch.tensor(0.0) # No loss if no routing happened

        # Calculate the fraction of tokens dispatched to each expert.
        # Shape: [N]
        tokens_per_expert = torch.mean(expert_mask.float(), dim=0)
        # Calculate the average router probability for each expert.
        # Shape: [N]
        router_prob_per_expert = torch.mean(router_probs, dim=0)

        # The loss is the dot product of these two quantities, scaled by the number of experts.
        auxiliary_loss = self.num_experts * torch.sum(tokens_per_expert * router_prob_per_expert)
        return auxiliary_loss

    def forward(self, x_flat: torch.Tensor) → tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        """
        Computes expert assignments and gating weights for a batch of tokens.
        Args:
            x_flat: The flattened input token embeddings. Shape: [T, D_in] (num_tokens, input_dim).

        Returns:
            tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
                top_k_weights: The weights for the top-k experts. Shape: [T, k]
                top_k_indices: The indices of the top-k experts. Shape: [T, k]
                auxiliary_loss: The scalar load-balancing loss.
        """
        # Compute router logits and probabilities
        logits = self.gating_network(x_flat)
        router_probs = F.softmax(logits, dim=-1)

        # Select the top-k experts for each token
        top_k_weights, top_k_indices = torch.topk(router_probs, k=self.k, dim=-1)

        # Create a mask for the auxiliary loss calculation
        expert_mask = F.one_hot(top_k_indices, self.num_experts).sum(dim=1)

        # Compute the auxiliary loss
        auxiliary_loss = self.compute_auxiliary_loss(router_probs, expert_mask)

        return top_k_weights, top_k_indices, auxiliary_loss

```

```

class BatchedLoRA(nn.Module):
    """
    Implements a batched version of Low-Rank Adaptation (LoRA) for Mixture-of-Experts.

    This module holds N sets of LoRA A and B matrices and computes the LoRA update
    for all N experts in parallel for a given input batch.
    """

    def __init__(self, input_dim: int, output_dim: int, rank: int, alpha: int, num_experts: int):
        """
        Initializes the BatchedLoRA layer.
        Args:
            input_dim: Input dimension (D_in) of the layer being adapted.
            output_dim: Output dimension (D_out) of the layer being adapted.
            rank: The rank 'r' of the LoRA decomposition.
            alpha: The LoRA scaling factor.
            num_experts: The total number of expert LoRA matrices (N).
        """
        super().__init__()
        # Define trainable LoRA parameters for all experts
        self.A_weights = nn.Parameter(torch.empty(num_experts, rank, input_dim))
        self.B_weights = nn.Parameter(torch.empty(num_experts, output_dim, rank))

        # LoRA scaling factor
        self.scaling = alpha/rank

        self.reset_parameters()

    def reset_parameters(self):
        """Initializes the LoRA matrices"""
        # A weights are initialized with kaiming uniform.
        # a is chosen based on the same initialization bound of a standard nn.Linear layer
        nn.init.kaiming_uniform_(self.A_weights, a=math.sqrt(5))
        # B weights are initialized with zero matrices
        nn.init.zeros_(self.B_weights)

    def forward(self, x: torch.Tensor) → torch.Tensor:
        """
        Computes the LoRA update delta for all experts in parallel.

        Args:
            x: Input tensor flattened to token level. Shape: [T, D_in] (num_tokens, input_dim)

        Returns:
            torch.Tensor: The LoRA update deltas for all N experts and T tokens.
                Shape: [T, N, D_out] (num_tokens, num_experts, Output_Dim)
        """
        # Down-projection: Applies all N expert A matrices to T tokens
        # 'td, nrd → tnr' maps [T, D_in] and [N, r, D_in] to [T, N, r]
        a_downproject = torch.einsum('td, nrd → tnr', x, self.A_weights)

        # Up-projection: Applies all N expert B matrices
        # 'tnr, nor → tno' maps [T, N, r] and [N, D_out, r] to [T, N, D_out]
        b_upproject = torch.einsum('tnr, nor → tno', a_downproject, self.B_weights)

        return b_upproject * self.scaling

```

```

class MoE_LoRA(nn.Module):
    """
    Implements a single attention block layer modified with a Mixture-of-Experts (MoE) architecture. The experts are low-rank (LoRA) adapters.

    This implementation uses a vectorized compute-all then select design. All experts are applied to all tokens, computed in parallel,
    then combined via a weighted sum from the router that acts as a gating mechanism.
    """
    def __init__(self, base_layer: nn.Module, num_experts: int, lora_rank: int, lora_alpha: int, k: int = 2):
        """
        Initializes the MoE-LoRA layer.

        Args:
            base_layer: The original PyTorch module (e.g., a ViT Attention block)
                        that this layer replaces and whose parameters will be frozen.
            num_experts: The total number of expert networks (N) in the pool. (Each one is a set of A and B LoRA parameters)
            lora_rank: The rank 'r' for the low-rank decomposition in LoRA experts.
            lora_alpha: The LoRA scaling factor (alpha).
            k: The number of experts to route each token to. Defaults to 2.
        """
        super().__init__()
        ### Original attributes of the modified block
        self.base_layer = base_layer
        self.num_heads = base_layer.num_heads
        self.scale = base_layer.scale
        self.attn_drop = base_layer.attn_drop
        self.proj_drop = base_layer.proj_drop

        ### Original qkv and projection of the modified block
        self.frozen_qkv = base_layer.qkv
        self.frozen_proj = base_layer.proj

        ### MoE components
        self.num_experts = num_experts
        input_dim = self.base_layer.qkv.in_features
        output_dim = self.base_layer.qkv.out_features
        self.router = Router(input_dim, num_experts, k)
        self.experts = BatchedLoRA(
            num_experts=num_experts,
            input_dim=input_dim,
            output_dim=output_dim,
            rank=lora_rank,
            alpha=lora_alpha,
        )

    def forward(self, x: torch.Tensor, **kwargs) -> torch.Tensor:
        """
        Defines the shape and data flow for the MoE-LoRA layer.

        Args:
            x (torch.Tensor): Input tensor.
                Shape: [B, S, D_in] (Batch, Sequence, Input Dim)
            **kwargs: Accepts additional keyword arguments for compatibility with
                parent modules, though they are not used in this layer.

        Returns:
            torch.Tensor: Output tensor of the attention block.
                Shape: [B, S, D_in] (Batch, Sequence, Input Dim)
        """
        ### 1. Setup and dimension definitions
        batch_size, seq_len, input_dim = x.shape
        num_tokens = batch_size * seq_len
        x_flat = x.view(num_tokens, input_dim)

        ### 2. Parallel forward passes
        # 2a. Main path (Frozen base QKV projection)
        qkv_base = self.frozen_qkv(x) # shape = [B, S, D_in]

        # 2b. Parallel MoE-LoRA path
        # Outputs: top-k weights [T, k], top-k indices [T, k], scalar loss
        top_k_weights, top_k_indices, auxiliary_loss = self.router(x_flat)
        # Store the auxiliary loss for forward hook collection
        self.aux_loss = auxiliary_loss

        # Compute all expert outputs (projects D_in -> 3*D_in (D_out))
        expert_output = self.experts(x_flat) # shape = [T, N, 3*D_in]

        ### 3. Gating and combination
        # Create sparse dispatch weights [T, N] from top-k outputs
        routing_matrix = F.one_hot(top_k_indices, self.num_experts) #Shape = [T, k, N]

        # Apply routing weights via broadcasting = [T, k, N] * [T, k, 1] -> [T, k, N]
        scaled_routing_matrix = routing_matrix * top_k_weights.unsqueeze(dim=-1)
        dispatch_tensor = scaled_routing_matrix.sum(dim=1) #Shape = [T, N]

        # Select and combine expert outputs via weighted sum using einsum
        # 'tno, tn -> to' combines [T, N, D_out] and [T, N] into [T, D_out]
        sparse_scaled_expert_output = torch.einsum('tno, tn -> to', expert_output, dispatch_tensor)

        ### 4. Add LoRA delta from the LoRA path to the frozen base QKV projection
        qkv = qkv_base + sparse_scaled_expert_output.view(batch_size, seq_len, -1)

        ### 5. Standard Multi-Head attention logic
        qkv = qkv.reshape(batch_size, seq_len, 3, self.num_heads, input_dim // self.num_heads).permute(2, 0, 3, 1, 4)
        q, k, v = qkv.unbind(0)
        attn = (q@k.transpose(-2,-1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)
        x = (attn @ v).transpose(1, 2).reshape(batch_size, seq_len, input_dim)
        x = self.frozen_proj(x)
        x = self.proj_drop(x)

        return x

```

```

class ModifiedModel(nn.Module):
    """
    The top-level class responsible for orchestrating the components.
    """
    def __init__(self, base_model: nn.Module, num_classes: int, model_config: dict):
        """
        Initializes the ModifiedModel class.
        Args:
            base_model: The pre-trained base model to be modified (either a DeiT or ViT).
            num_classes: Number of classes in the dataset.
            model_config: The model configuration dictionary taken from the config files.
        """
        super().__init__()
        self.base_model = base_model

        mode = model_config.get('mode')
        lora_rank = model_config.get('lora_rank')
        lora_alpha = model_config.get('lora_alpha')
        num_experts = model_config.get('num_experts')
        k = model_config.get('k')
        target_blocks_indices = model_config.get('target_blocks_indices', [])

        # freeze all parameters first
        for param in self.base_model.parameters():
            param.requires_grad = False

        # Head replacement
        in_features = self.base_model.head.in_features
        self.base_model.head = nn.Linear(in_features, num_classes)

        # Conditional setup based on mode
        if mode == 'full_finetune':
            # Unfreeze everything
            for param in self.base_model.parameters():
                param.requires_grad = True

        elif mode == 'linear_probe':
            for param in self.base_model.head.parameters():
                param.requires_grad = True

        elif mode == 'lora':
            target_modules = [f"blocks.{i}.attn.qkv" for i in target_blocks_indices]
            lora_config = LoraConfig(r=lora_rank, lora_alpha=lora_alpha, target_modules=target_modules)
            self.base_model = get_peft_model(self.base_model, lora_config)
            for param in self.base_model.head.parameters():
                param.requires_grad = True

        elif mode == 'moe_lora':
            for block_index in target_blocks_indices:
                original_attention_layer = self.base_model.blocks[block_index].attn
                self.base_model.blocks[block_index].attn = MoE_LoRA(
                    base_layer=original_attention_layer,
                    num_experts=num_experts,
                    lora_rank=lora_rank,
                    lora_alpha=lora_alpha,
                    k=k
                )
            for param in self.base_model.head.parameters():
                param.requires_grad = True
        else:
            raise ValueError(f"Unknown mode: {mode}")

        self.collected_auxiliary_loss = [] # store auxiliary loss for the forward hook to gather

    def collect_auxiliary_loss(module, _, __):
        """
        The forward hook function to collect the stateful auxiliary loss from an nn.Module.
        Args:
            module: The module which would have its auxiliary loss collected.
            _: Ignored input
            __: Ignored output
        """
        if hasattr(module, 'aux_loss'):
            self.collected_auxiliary_loss.append(module.aux_loss)

    for block in self.base_model.blocks:
        if isinstance(block.attn, MoE_LoRA):
            block.attn.register_forward_hook(collect_auxiliary_loss)

    def forward(self, x: torch.Tensor) → tuple[torch.Tensor, torch.Tensor]:
        """
        Defines the forward pass for the entire model.
        Args:
            x: The raw input tensor.

        Returns:
            logits: The logits of the model.
            total_aux_loss: The auxiliary loss (if it uses MoE_LoRA). Else, 0.
        """
        self.collected_auxiliary_loss.clear()
        logits = self.base_model(x)

        # Conditional check for if an auxiliary loss exists. If not, pass 0 as the auxiliary loss.
        if self.collected_auxiliary_loss:
            total_aux_loss = torch.stack(self.collected_auxiliary_loss).mean()
        else:
            total_aux_loss = torch.tensor(0.0, device=logits.device)

        return logits, total_aux_loss

```